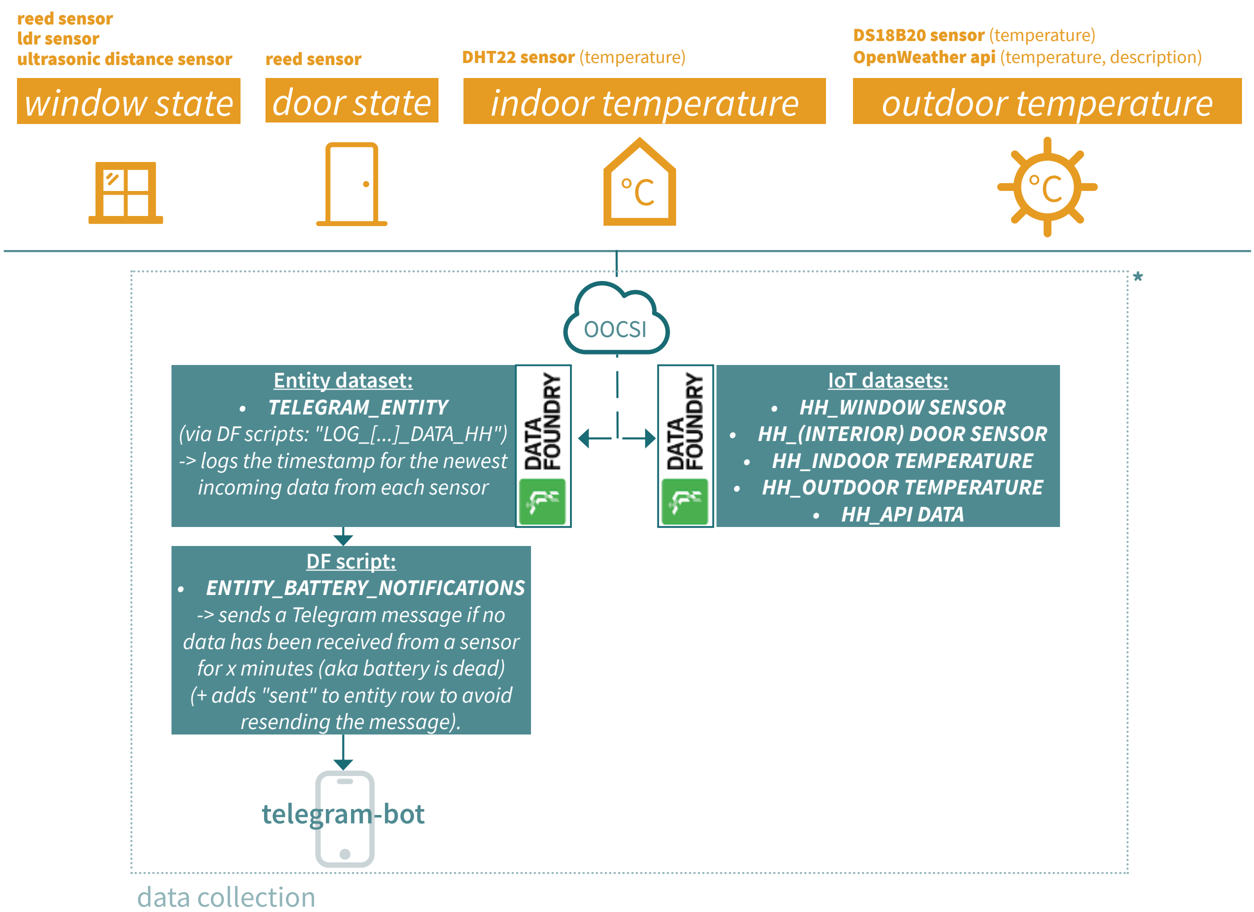
COOL AI: technical details

This document explains the technical details for the Cool AI study setup; how data is sent from and to DataFoundry, to Python notebooks and scripts, and to the interface. On page 1 I explain the data collection part of the project, on page 2 the data processing part, and on page 3 you will find the full diagram (data collection + data processing).

1. Data collection (sensors)

Data is collected at the participants' home by four kinds of sensor devices: (1) window (reed sensor, light sensor, ultrasonic sensor); (2) door (reed sensor); (3) indoor temperature (DHT22), and (4) outdoor temperature (DS18B20).

These devices are all controlled by an esp32 microcontroller and send data via OOCSI to their resp. IoT databases in DataFoundry. An entity dataset also logs the last timestamps for incoming data from each sensor to monitor whether the battery is still live (& send a Telegram message to the participant to charge the battery in case of dead battery).



* (The DataFoundry project also contains "PILOT" versions of each dataset and script for testing purposes)

2. Data processing (to interface)

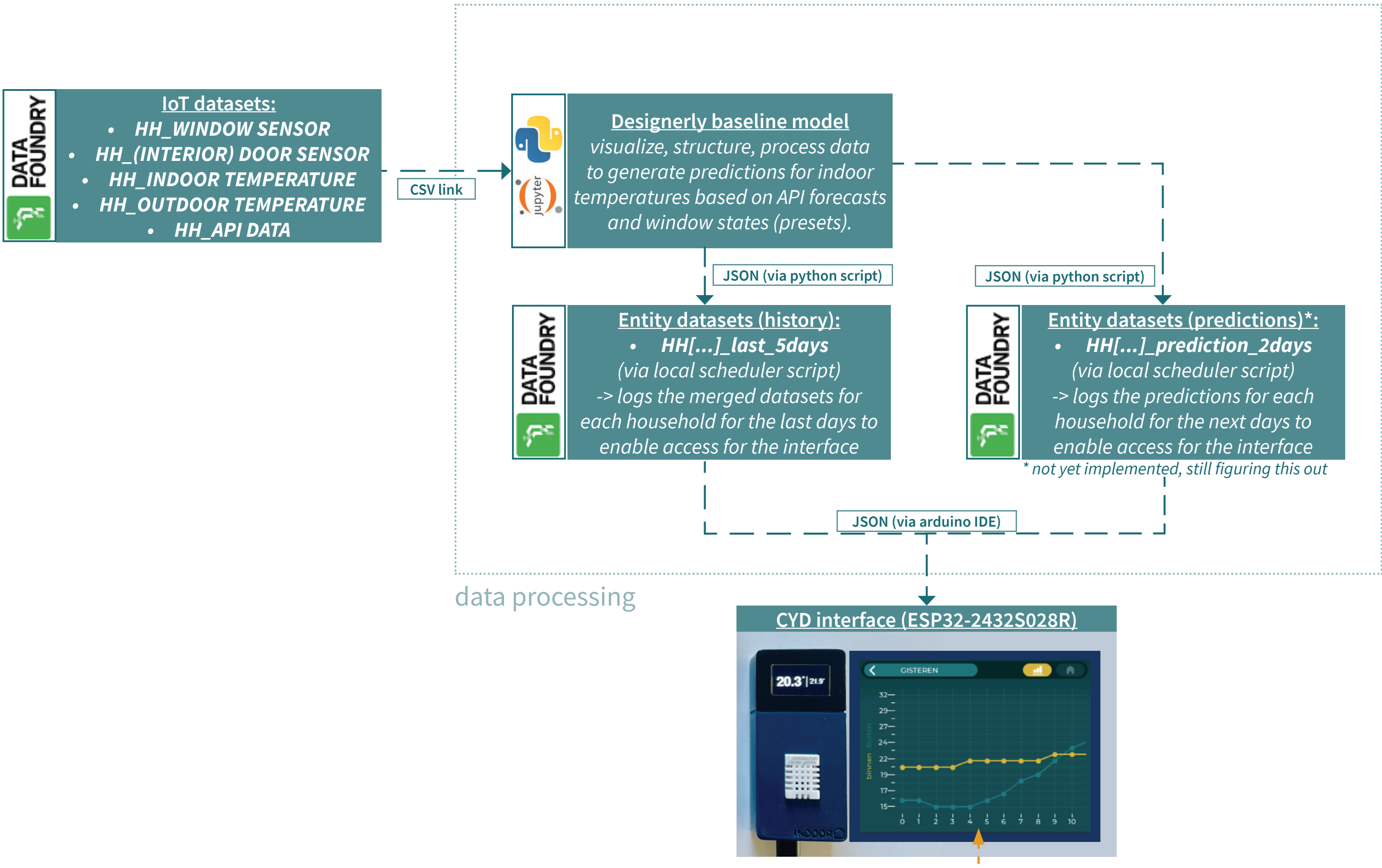
Data from the IoT datasets is accessed via their CSV links in Python Notebooks to visualize, structure, and process the data to create a ‘Designerly Baseline Model’ that generates predictions for indoor temperatures based on forecast (API) data and (preset) window states.

The output from the resulting Python scripts is sent back into (Entity) datasets in DataFoundry, so that the data can be accessed as a JSON by the CYD interface (ESP32-2432S028R). The Python scripts are run at fixed time intervals by a scheduler (local).

The interface has two tabs: “history” and “predictions”:

1. History tab: A python script merges the datasets and sends the last 5 days of relevant data back to an Entity dataset in DataFoundry for each participant. This entity dataset can then be accessed (as a JSON) by the CYD to display this data in the history tab.
2. Predictions tab*: A python script processes the datasets and generates indoor temperature predictions for each window/curtain/shading state based on API forecast data. This results in multiple datasets (per window/curtain/shading state). These datasets are sent back to (Entity) datasets in DataFoundry so they can be accessed (as JSON) by the CYD to display the predictions and change them according to user input (changing the window/curtain/shading states).

** This part has not been implemented yet, and I am still figuring out how exactly to do this (unsure if this approach will be the final approach).*



The CYD uses the LGVL library in Arduino IDE to create the GUI.
(+ I use SquareLine to design the GUI)

3. Full diagram